

DT-1 — DATA MODELING (FINAL)

Document Processing Analytics Platform

Focus: Load Reporting + Pipeline Health + Invoice Extraction | Multi-tenant, Role-based | Queries respond in 3–5 seconds

This document is the FINAL, locked data model for DT-1. It answers all 5 required data-modeling questions plus the remediation feature and confidence logic. Use it as the basis for DT-2 (API Design).

Overview — 11 Tables across 4 Layers

- LAYER 1 • IDENTITY → Tenants • Sites • Users • UserSiteAccess
- LAYER 2 • PIPELINE → Transactions • Files • FileStepHistory
- LAYER 3 • SUPPORT → ErrorCatalog • ActivityLog
- LAYER 4 • EXTRACTION → DocumentTypes • ExtractedFields

Layer 1 — Identity (who & access)

1. TENANTS — customer companies

Column	Type	Constraint
id	UUID	PK
name	VARCHAR(100)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL
is_active	BOOLEAN	NOT NULL

2. SITES — physical locations of a tenant

Column	Type	Constraint
id	UUID	PK
tenant_id	UUID	FK → Tenants
name	VARCHAR(100)	NOT NULL
location	VARCHAR(200)	NULLABLE
created_at	TIMESTAMPTZ	NOT NULL
is_active	BOOLEAN	NOT NULL

3. USERS — dashboard logins

Column	Type	Constraint
id	UUID	PK
tenant_id	UUID	FK → Tenants
email	VARCHAR(200)	UNIQUE
password_hash	VARCHAR(500)	NOT NULL
role	VARCHAR(50)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL
is_active	BOOLEAN	NOT NULL

4. USER_SITE_ACCESS — bridge (which user sees which site)

Column	Type	Constraint
id	UUID	PK
user_id	UUID	FK → Users
site_id	UUID	FK → Sites
granted_at	TIMESTAMPTZ	NOT NULL
(unique)		UNIQUE(user_id, site_id)

Layer 2 — Pipeline (the core)

5. TRANSACTIONS — the "TId" rows on the Load Reporting dashboard

Column	Type	Constraint
id	UUID	PK (the TId)
tenant_id	UUID	FK → Tenants
site_id	UUID	FK → Sites
state	VARCHAR(20)	NOT NULL
source_system	VARCHAR(100)	NOT NULL
total_files	INTEGER	NOT NULL
uploaded_count	INTEGER	NOT NULL
processing_count	INTEGER	NOT NULL
failed_count	INTEGER	NOT NULL
completed_count	INTEGER	NOT NULL
submitted_at	TIMESTAMPTZ	NOT NULL
last_updated_at	TIMESTAMPTZ	NOT NULL
completed_at	TIMESTAMPTZ	NULLABLE

The 4 counters are pre-aggregated so the dashboard loads instantly (no counting 1M rows live).

6. FILES — generic file records (~1M rows)

Column	Type	Constraint
id	UUID	PK
tenant_id	UUID	FK → Tenants
site_id	UUID	FK → Sites
transaction_id	UUID	FK → Transactions
document_type_id	UUID	FK → DocumentTypes (NULLABLE)
file_name	VARCHAR(255)	NOT NULL
file_type	VARCHAR(50)	NOT NULL
status	VARCHAR(20)	NOT NULL
current_step	VARCHAR(50)	NOT NULL
file_size_bytes	BIGINT	NULLABLE
extraction_status	VARCHAR(20)	NULLABLE
extraction_confidence	DECIMAL(4,3)	NULLABLE
last_updated_at	TIMESTAMPTZ	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL

7. FILE_STEP_HISTORY — step-by-step audit per file

Column	Type	Constraint	Note
id	UUID	PK	

file_id	UUID	FK → Files	
document_type_id	UUID	FK → DocumentTypes (NULLABLE)	NEW – denormalized for fast error panel
step_name	VARCHAR(50)	NOT NULL	Upload/Validate/Transform/Load
status	VARCHAR(20)	NOT NULL	
started_at	TIMESTAMPTZ	NULLABLE	
completed_at	TIMESTAMPTZ	NULLABLE	
error_code	VARCHAR(50)	NULLABLE	links to ErrorCatalog
error_message	TEXT	NULLABLE	

Decision: one row per step (not a JSON array) → so steps are queryable, sortable, groupable & indexable.

Layer 3 — Support

8. ERROR_CATALOG — known errors + how to fix them

Column	Type	Constraint
id	UUID	PK
error_code	VARCHAR(50)	UNIQUE
description	TEXT	NOT NULL
remediation_msg	TEXT	NULLABLE
created_at	TIMESTAMPTZ	NOT NULL
updated_at	TIMESTAMPTZ	NOT NULL

Powers the remediation popup — turns ERR_BAD_SCHEMA into "Check your column headers."

9. ACTIVITY_LOG — append-only system audit trail

Column	Type	Constraint
id	UUID	PK
tenant_id	UUID	FK → Tenants
site_id	UUID	FK → Sites
event_type	VARCHAR(100)	NOT NULL
entity_type	VARCHAR(50)	NOT NULL
entity_id	UUID	NOT NULL
entity_name	VARCHAR(255)	NULLABLE
old_state	VARCHAR(50)	NULLABLE
new_state	VARCHAR(50)	NULLABLE
triggered_by	VARCHAR(100)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL

Layer 4 — Extraction

10. DOCUMENT_TYPES — catalog of supported types (extensible)

Column	Type	Constraint
id	UUID	PK
type_name	VARCHAR(100)	UNIQUE
category	VARCHAR(10)	NOT NULL (PDF/CSV)
description	TEXT	NULLABLE
is_active	BOOLEAN	NOT NULL

created_at	TIMESTAMPTZ	NOT NULL
------------	-------------	----------

11. EXTRACTED_FIELDS — flexible key-value extraction (EAV pattern)

Column	Type	Constraint
id	UUID	PK
file_id	UUID	FK → Files
tenant_id	UUID	FK → Tenants
site_id	UUID	FK → Sites
field_name	VARCHAR(100)	NOT NULL
field_value	TEXT	NULLABLE
confidence	DECIMAL(4,3)	NULLABLE
is_valid	BOOLEAN	NOT NULL
extracted_at	TIMESTAMPTZ	NOT NULL

EAV = any new doc type stores its fields as rows here → ZERO schema migrations ever.

Relationships

From → To	Type
Tenants → Sites	1:N
Tenants → Users	1:N
Sites → Transactions	1:N
Transactions → Files	1:N
Files → FileStepHistory	1:N
Files → ExtractedFields	1:N
DocumentTypes → Files	1:N
DocumentTypes → FileStepHistory	1:N
Users ↔ Sites (via UserSiteAccess)	N:M
FileStepHistory → ErrorCatalog (by error_code)	soft ref

Indexes (why each exists)

Table	Index	Purpose
TRANSACTIONS	(tenant_id, site_id, last_updated_at DESC)	Main load-reporting view
TRANSACTIONS	(tenant_id, site_id, state)	Filter by state
FILES	(transaction_id)	Drill-down: files in a transaction
FILES	(tenant_id, site_id, status, last_updated_at DESC)	Recent failures
FILES	(tenant_id, site_id, document_type_id)	"Show all invoices"
FILE_STEP_HISTORY	(file_id)	File step drill-down + log download
FILE_STEP_HISTORY	(step_name, status)	Error analysis by step
EXTRACTED_FIELDS	(file_id)	Load a file's fields
EXTRACTED_FIELDS	(tenant_id, site_id, field_name)	Query by field
ACTIVITY_LOG	(tenant_id, site_id, created_at DESC)	Activity feed
USER_SITE_ACCESS	(user_id, site_id)	Auth check

Tenant Isolation (3 layers)

1. Every pipeline/support/extraction table carries tenant_id + site_id
2. JWT token holds { tenantId, siteId, role } – set at login
3. EF Core Global Query Filter auto-injects
WHERE tenant_id = X AND site_id = Y on EVERY query

```
modelBuilder.Entity<File>()
    .HasQueryFilter(f => f.TenantId == _currentUser.TenantId
        && f.SiteId == _currentUser.SiteId);
```

Even if a dev forgets a WHERE clause, the filter protects the data. Customer A can never see Customer B.

Key Business Rule — "Any file fails → transaction fails"

On any file status change:

1. Update the 4 counters on its Transaction
2. Recompute Transaction.state:
 - failed_count > 0 → Failed
 - else uploaded_count + processing_count > 0 → Processing
 - else → Completed
3. Update last_updated_at
4. Write an ActivityLog entry

Failed-Files + Remediation Feature

The dashboard shows a Failed Files panel with columns: File Name | Error | 3-dots menu. Clicking the 3-dots menu reveals two actions:

- View Remediation — looked up from ErrorCatalog by error_code (turns ERR_BAD_SCHEMA into a human fix message).
- Download Logs — built from that file's FileStepHistory rows (full step-by-step trace).

Uses 3 tables together: Files (name) + FileStepHistory (error & steps) + ErrorCatalog (the fix).

What's shown	Source table	Column
File Name	Files	file_name
Error code	FileStepHistory	error_code
Remediation message	ErrorCatalog	remediation_msg (by error_code)
Download Logs	FileStepHistory	all step rows for that file

Confidence — how it is determined

PdfPig reads the real text layer (no built-in score), so confidence is derived from how cleanly we matched the value:

How value was found	Confidence
Clean regex / keyword match	0.95
Found via fallback guess	0.60
Not found	0.0

"Confidence reflects how reliable the extraction method was. If we later use OCR for scanned docs, we would store the OCR engine's real score instead."

Key Design Decisions

- UUID PKs → globally unique
- VARCHAR status → add new statuses without migrations
- Pre-aggregated counters → dashboard under 3–5s
- Denormalized tenant_id/site_id on Files → no JOIN on 1M rows
- doc_type_id on FileStepHistory → fast error panel without extra JOIN
- EAV (ExtractedFields) → unlimited doc types, zero migrations
- One row per step → queryable, indexable history
- Append-only ActivityLog → tamper-evident audit
- EF Core Migrations → version-controlled schema

Hand-off Notes for DT-2 (API Design)

With the schema locked, DT-2 should define endpoints around these core flows:

- Auth/login → issue JWT carrying { tenantId, siteId, role }.
- GET load-reporting list → paginated Transactions with the 4 counters + state + last_updated.
- GET transaction drill-down → list of Files in a transaction.
- GET file drill-down → FileStepHistory rows for a file.
- GET failed-files for a transaction → file name + error_code.
- GET remediation for a file → joins ErrorCatalog by error_code.
- GET file logs → downloadable log built from FileStepHistory.
- GET extracted fields for a file → rows from ExtractedFields.
- GET activity log → paginated, filtered by tenant/site.

All endpoints must respect tenant isolation (filter by tenant_id + site_id from the JWT) and target the 3–5s response budget using the indexes above.

— END OF DT-1 (FINAL) —